

Unidad 6

Colecciones

Contenido

2

Contenido:

- ▶ Tipos parametrizados o genéricos
- ▶ Interfaz **Collection**
- ▶ Interfaz **List**
- ▶ Interfaz **Set**
- ▶ Conversiones entre colecciones
- ▶ Clase **Collections**
- ▶ Interfaz **Map**

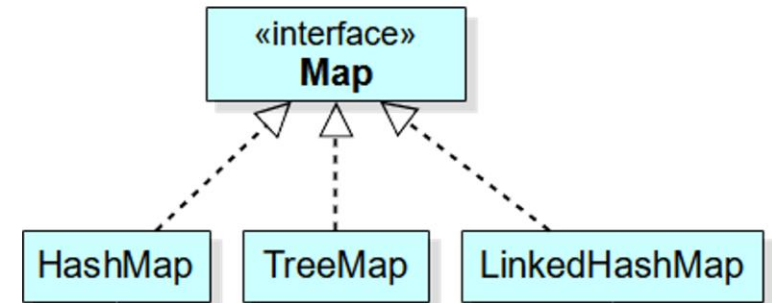
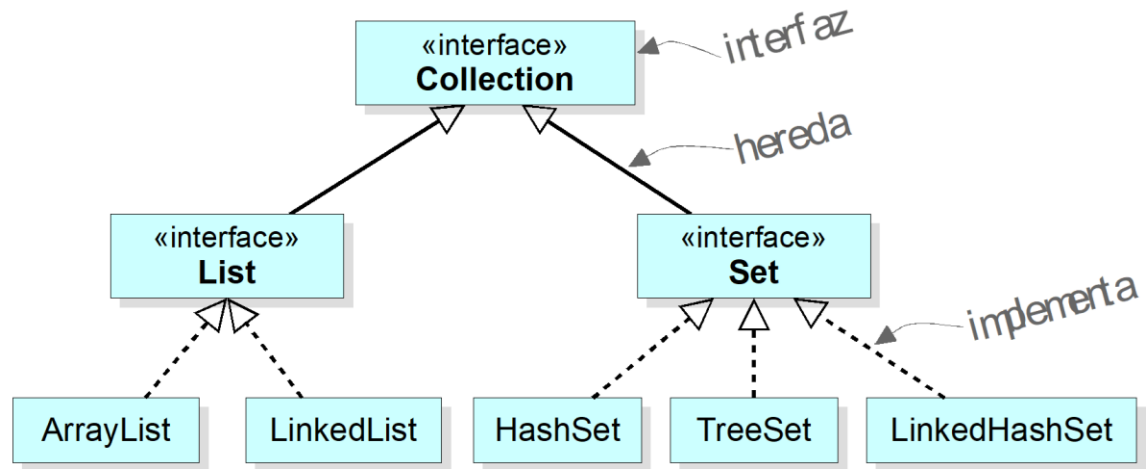
- ▶ Hasta ahora conocemos la estructura de datos **array** que permite almacenar en una variable **varios valores** de un determinado tipo de datos.
- ▶ Java nos ofrece una posibilidad más flexible para cubrir esta necesidad con **estructuras dinámicas** que pueden variar su tamaño **adaptándose** a lo que se requiera **en tiempo de ejecución**:
 - ▶ **Estructuras dinámicas** que implementan la **interfaz Collection**. Es decir, java dispone de diferentes estructuras de datos que comparten un conjunto de métodos declarados en la interfaz Collection que sirven para operar con las colecciones de datos. Cada tipo de dichas estructuras implementa los métodos propios de una colección de forma distinta:
 - Inserción
 - Eliminación
 - Ordenación
 - Búsqueda

- ▶ Una **colección** es un objeto que agrupa un **conjunto de elementos** que generalmente guardan relación entre ellos.
- ▶ En java se definen diferentes tipos de colecciones dependiendo de qué se quiere representar y hacer con el conjunto de elementos. Cada tipo tiene sus características específicas. Debido a esta 'diversificación' , Collection es una interfaz (con su jerarquía asociada) y no una clase.
- ▶ El **framework JCF** (Java Collection Framework) consiste en una **estructura jerárquica de interfaces**, implementadas en diversas **clases** y con un conjunto de **algoritmos** para operar con los conjuntos de elementos.
- ▶ JFC se encuentra en java.util

- ▶ Tipos de colecciones:
 - ▶ **Listas** (Interfaz List): Colección en la que **el orden es relevante** que permite elementos **duplicados**
 - ▶ **Conjuntos** (Interfaz Set): Colección en la que el **orden no es relevante** y que **no permite** elementos **duplicados**
 - ▶ **Mapas** o Diccionarios:
 - Se utilizan para guardar datos identificados por claves que no se repiten.
 - Implementan la interfaz **Map** que **no hereda** de Collection

Introducción

6



- ▶ Ventajas del uso de colecciones:
 - ▶ Tamaño dinámico de las estructuras de datos
 - ▶ Métodos ya implementados para operaciones habituales : buscar, añadir, eliminar...
 - ▶ Facilitan la definición de estructuras de datos complejas

Tipos parametrizados o genéricos

8

► Clases con parámetros genéricos

- Se utilizan cuando se necesita definir atributos en la clase que puedan ser de cualquier tipo
- Con el '**operador diamante**' <> se señala la utilización de **tipos genéricos** (generics)
- Con la expresión **class Prueba<T>** se representa que en la clase Prueba se utiliza el tipo de datos genérico T en declaraciones concretas.
- T debe ser una **interfaz o clase**, nunca un tipo primitivo (int, double..)
- El **compilador comprueba** los tipos que tienen los valores utilizados en la clase

Tipos parametrizados o genéricos

9

► Clases con parámetros genéricos (II)

- Se suele utilizar T para el tipo genérico.
 - E para elementos de colecciones
 - K para claves
 - V para valores
 - N para números
- Las clases definidas con tipos genéricos también se pueden utilizar sin ellos en cuyo caso el compilador trabaja con variables de tipo Object. No se comprueban los tipos y se pueden guardar objetos de distintas clases mezclados.

Ej. EjemploContenedor

- ▶ List es una colección que permite duplicados, en la que importa el orden.
- ▶ Cada elemento tiene una posición (índice)
- ▶ Implementaciones:
 - ▶ **ArrayList**: Similar a los arrays. Adecuado cuando se realizan más búsquedas y accesos por índice que escrituras.
 - ▶ **LinkedList**: Lista doblemente enlazada. Cada elemento se enlaza con el anterior y el siguiente. Adecuada cuando se realizan inserciones/eliminaciones en posiciones intermedias.
- ▶ La sintaxis para construir un objeto ArrayList(LinkedList) con un tipo genérico de datos es:

```
ArrayList <E> lista = new ArrayList<>(); // solo se pueden insertar elementos de tipo E
```

```
List <E> lista = new ArrayList<>();
```

► Métodos principales.

- **add**(elemento) -> Agrega un elemento al final de la lista.
- **add**(pos, elemento) -> Inserta un elemento en una posición específica.
- **remove**(elemento) -> Elimina la primera aparición del elemento.
- **remove**(pos) -> Elimina el elemento en esa posición
- **get**(pos) -> Devuelve el elemento de la posición 'pos'.
- **set**(pos, nuevoValor) -> Reemplaza el elemento en la posición 'pos' por 'nuevoValor'.
- **contains**(elemento) -> Devuelve true si el elemento está en la lista.
- **size**() -> Devuelve el número de elementos de la lista.
- **isEmpty**() -> Devuelve true si la lista está vacía.
- **clear**() -> Elimina todos los elementos de la lista.

- ▶ Un **iterador** es un objeto que permite recorrer de forma segura una colección elemento a elemento sin necesidad de conocer el detalle de la implementación (listas, conjuntos,...)
- ▶ Con un iterador es posible eliminar elementos de la colección mientras se está recorriendo (a diferencia del foreach)
- ▶ Para utilizar un iterador el primer paso es crearlo: El método

Iterator<E> iterator()

invocado por la colección, nos devuelve un iterador asociado a ella.

Aquí E será del mismo tipo que el de la colección.

- ▶ Ejemplo: Dada una colección de clientes, el iterador se crea con
`Iterator<Cliente> it = coleccionClie.iterator();`
`it` es el iterador a utilizar para recorrer la colección 'coleccionClie'
- ▶ Para recorrer una colección con un iterador disponemos de 2 métodos que se complementan y utilizan conjuntamente:
 - ▶ **boolean hasNext():** comprueba si hay un elemento siguiente, es decir, si quedan elementos por visitar, y devuelve true o false, según el caso.
 - ▶ **E next():** el iterador avanza al siguiente elemento, si existe, y lo devuelve.

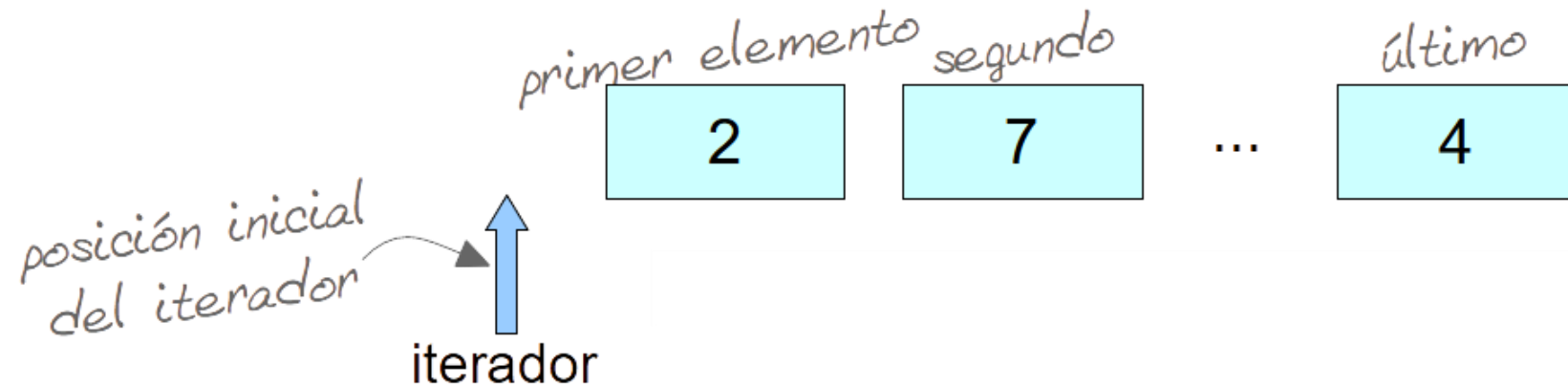
En caso de que no haya siguiente, porque estemos al final de la colección o porque esta estuviera vacía, next() lanzará la **excepción NoSuchElementException**.

La primera llamada a next() devuelve el primer elemento de la colección si esta no está vacía.

Iteradores

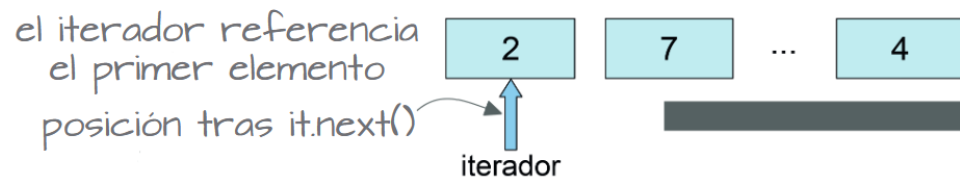
14

```
Iterator it = coleccion.iterator();
```



- En la práctica para evitar la excepción se utiliza hasNext() antes de llamar a next().

```
Iterator<Cliente> it = coleccionClie.iterator();  
for ( ; it.hasNext(); ) {  
    Cliente p = it.next();  
    System.out.println(p);  
}
```



- La declaración e inicialización del iterador it se podría haber incluido en la parte de inicialización del bucle for.
- La parte de los incrementos está vacía. Esto se debe a que el método next() incrementa automáticamente el iterador para que apunte al siguiente elemento y después lo devuelve.

- ▶ Desde un iterador es posible eliminar elementos de una colección:
- ▶ **void remove():** Elimina de la colección el **último elemento devuelto por next()**, que es el apuntado por el iterador en cada momento.
- ▶ Así podemos eliminar un elemento de una colección dependiendo de alguna condición **mientras se está recorriendo con un iterador**.

Esta es la única forma de eliminar un elemento preservando la integridad de la colección.

No confundir con el método `remove(Object ob)` de la interfaz `Collection`, que se invoca desde un objeto colección y no debe usarse dentro del bucle de un iterador. En este contexto, debemos invocar el del iterador, al que no se le pasa ningún parámetro.

► Ejemplo:

```
Iterator<Cliente> it = coleccionClie.iterator();
while(it.hasNext()) {
    Cliente p = it.next();
    if (p.nombre.equals("pepe")){
        it.remove(); /*elimina p, ultimo cliente devuelto por next() ¡No usar
                     coleccionClie.remove(p)!*/
    }
}
```

► Recordatorio: para recorrer una colección (foreach)

```
for( Cliente c : coleccionClie) { ....}
```

Interfaces Comparable y Comparator

18

- ▶ Las interfaces **Comparable** y **Comparator** se utilizan para definir criterios de ordenación de objetos.
- ▶ Son útiles para **ordenar elementos** en las colecciones, por ejemplo, `Collections.sort()`, `Arrays.sort()`.
- ▶ **Interfaz Comparable<T> :**
 - Orden fijo y natural de los objetos
 - Se implementa **dentro de la clase del objeto** a ordenar. Define un orden natural (por ejemplo: por nombre, ID, etc.).
- ▶ **Interfaz Comparator<T> :**
 - Orden externo (más flexible) o múltiple
 - Se utiliza para definir orden **fuera de la clase**, personalizado, sin necesidad de modificar la propia clase.

Interfaz Comparable<T>

19

- ▶ Declara el método **public int compareTo(T OtherObj)**
- ▶ Se **sobrescribe** en la clase que implementa la interfaz el método compareTo concretando con qué atributos y cómo se determina el orden natural de los objetos de la clase.
- ▶ compareTo debe retornar un valor:
 - ▶ =0 si *this* y *OtherObj* son iguales (respecto al criterio de orden).
 - ▶ <0 si *this* < *OtherObj*
 - ▶ >0 si *this* > *OtherObj*

Interfaz Comparable<T>

20

- ▶ Se utiliza para definir el orden natural que se aplicará por defecto a los objetos: por nombre, edad, ID, etc...
- ▶ Ejemplo de uso: en colecciones es automático para ordenaciones estándar: **Collections.sort()** y otras estructuras ordenadas (TreeSet, TreeMap)
- ▶ **Solo se puede definir una lógica de ordenación**, aunque esta puede ser compuesta (ej. orden por nombre y para mismo nombre orden por edad)
- ▶ Se debe **modificar la clase** para poder aplicar el orden con este método.

Interfaz Comparator<T>

21

- ▶ Declara el método **int compare(T obj1, T obj2);**
- ▶ Para utilizarlo se pasa como parámetro en métodos de ordenación (Collections.sort(personas, **new ComparadorPorEdad();**);)
- ▶ Permite la definición de varios criterios de ordenación, se puede crear un 'comparator' para cada criterio.
- ▶ Se declaran en una clase independiente, en la que se **sobrescribe 'compare'** accediendo a los atributos del objeto por el que se quiere establecer la comparación.
 - ▶ Retorna 0 si son iguales
 - ▶ Retorna <0 si el objeto actual es menor que el otro
 - ▶ Retorna >0 si el objeto actual es mayor que el otro

- ▶ Los conjuntos (**Set**) son colecciones que almacenan **elementos únicos**, es decir, no admiten duplicados.
 - ▶ Para detectar duplicados se utiliza el método **hashCode()** para agrupar objetos en la misma posición hash.
 - ▶ Si el hash coincide, se verifica la igualdad usando el método **equals()**.
- ▶ Al utilizar un Set **no se garantiza el orden**, este dependerá de la implementación
- ▶ Algunas implementaciones permiten el valor NULL (hashset)
- ▶ Implementaciones de la interfaz Set:
 - ▶ **HashSet**
 - ▶ **LinkedHashSet**
 - ▶ **TreeSet**

- ▶ **HashSet:**

- ▶ Basado en una tabla hash. (¡puede darse colisión!)
- ▶ No garantiza el orden de los elementos.
- ▶ Permite elementos null.

- ▶ **LinkedHashSet**

- ▶ Basado en una tabla hash con una lista vinculada para mantener el orden de inserción.
- ▶ Permite elemento null.

► Colisión en un HashSet:

- Después de identificar la colisión por el mismo valor hash, el HashSet compara el nuevo objeto con los ya presentes usando el método equals().
- Si equals() devuelve true, el conjunto asume que el objeto ya existe y no lo añade.
- Si equals() devuelve false, el objeto se considera distinto y se añade a la lista en esa posición hash.

► **TreeSet:**

- Basado en un árbol rojo-negro (estructura ordenada).
 - árbol binario de búsqueda autobalanceado
 - La raíz siempre es negra. Las hojas son negras (y no contienen datos : null).
 - Los hijos de un nodo rojo son siempre negros.
- Ordena los elementos de forma natural (alfabético para cadenas, numérico para enteros).
- No permite elementos null

- ▶ La interfaz **Map** define una estructura de datos que almacena pares clave-valor.
 - ▶ Cada valor está asociado a una clave única. No se permiten claves duplicadas. Si se intenta insertar una existente, se sobrescribe el valor anterior.
 - ▶ Sí se permiten valores duplicados
 - ▶ No hereda de Collection
 - ▶ Utiliza tipos genéricos (clave y valor) -> `Map<K,V>`

Interfaz Map

27

► Métodos más relevantes.

Método	Descripción
put(K clave, V valor)	Agrega o reemplaza una entrada.
get(Object clave)	Retorna el valor asociado a la clave.
containsKey(Object clave)	Devuelve true si la clave existe.
containsValue(Object valor)	Devuelve true si el valor existe.
remove(Object clave)	Elimina el par clave-valor.
keySet()	Retorna un Set con las claves.
values()	Retorna una colección de valores.
entrySet()	Retorna un conjunto de entradas (pares clave-valor).
size()	Número de pares clave-valor.
isEmpty()	Verifica si el Map está vacío.

- ▶ **HashMap.**
 - ▶ Es la implementación más utilizada
 - ▶ No garantiza el orden
 - ▶ Permite una clave null y varios valores null